

Smart Contract Residual Attention Net (SCRAN): A Hybrid Residual-Attention Ensemble Approach for Vulnerability Detection

Aman Sriven

Department of Computer Science and Engineering
Texas A&M University
College Station, TX, USA
aman.sriven@tamu.edu

Dr. Irfan Khan

Department of Computer Science and Engineering
Texas A&M University at Galveston
Galveston, TX, USA
irfankhan@tamu.edu

Abstract—Smart Contracts (SCs) have transformed blockchain technology ecosystems, significantly improving transparency and automation across various industries. However, their increasing complexity highlights the importance of security and vulnerability detection. This research explores the comprehensive BCCC-SCsVul-2024 dataset, which consists of 111,897 Solidity source code samples and 11 vulnerability classes. We created a model named Smart Contract Residual Attention Net (SCRAN), which uses feed-forward neural networks and has an architecture specifically designed for smart contract vulnerability detection. SCRAN integrates residual connections and hierarchical scanning for preserving deep features and addressing vanishing gradient issues, along with multi-head attention mechanisms for identifying important feature relationships. The attention mechanisms, focal loss parameters, and use of balanced threshold selection in the model help address the class imbalance, which is persistent throughout the dataset. The model is tailored for the common vulnerabilities (eg, reentrancy, overflow, access control). SCRAN employs an ensemble framework that combines the predictions of multiple models with varying hyperparameters for better generalization. The comprehensive experimental evaluations on the dataset demonstrate substantial improvements in precision and accuracy for vulnerability detection in the smart contracts dataset. SCRAN’s scalable and interpretable architecture makes it highly suitable for real-world smart contract security, including automated security auditing and development-time vulnerability detection.

Index Terms—Smart contracts, vulnerability detection, deep learning, attention, class imbalance, ensemble learning

I. INTRODUCTION

Smart contracts are deployed on countless blockchain applications and help enable trustworthy interactions in finance, gaming, and governance. However, they are immutable and can be prone to vulnerabilities and flaws in the code. When vulnerabilities are exploited, it can result in financial disasters like the DAO exploit in 2016, which resulted in a loss of \$60 million. This incident highlights the need for safety measures to be taken.

Detecting vulnerabilities in smart contracts isn’t just something you can solve overnight. Contracts exhibit various persisting vulnerability types, so the task is multi-label in nature.

In the real world, vulnerabilities don’t have a perfect ratio in which they exist, so models must learn to deal with class imbalance, as certain classes represent even less than 1% of samples. The feature space is also derived from control-flow graphs, opcode usage, and data dependencies, which is multi-dimensional and may be difficult to interpret without advanced analysis tools.

In the past, many static analysis tools like *Mythril* and *Slither* have been used, which rely on rule-based engines and symbolic execution. This offers interpretability, but struggles with complex semantic patterns and unseen attacks.

To address the limitations in the world of smart contract vulnerability detection, we propose our custom model: **Smart Contract Residual Attention Network (SCRAN)**. The model is an ensemble deep learning framework that is tailored for smart contract vulnerability detection, which can handle class imbalance. SCRAN combines advanced data augmentation (SMOTE + noise injection), residual connections, multi-head attention mechanisms, class-specific heads, and threshold tuning to optimize and increase the reliability of detection for the 11 different vulnerability types in the given dataset.

Key Features of the Model Include:

- Comprehensive data preprocessing pipeline including SMOTE-based oversampling, probabilistic augmentation, and dynamic class weighting.
- Novel neural architecture integrating residual connections, multi-head attention, and class-specific detectors to enhance contextual feature representation.
- Focal loss function with adaptive gamma and class-specific threshold optimization to target rare class performance.
- Ensemble framework that utilizes three SCRAN instances with different hyperparameter tuning and class-wise logit weighting for improved generalization and reliability
- Improvements of +3.46% macro F1 and +1.79% AUC on the BCCC-SCsVul-2024 dataset.

The rest of the paper is organized as follows: Section II reviews related work, Section III outlines the dataset and preprocessing, Section IV details SCRAN’s architecture and

training, Section V presents experimental results and ablation studies, and Section VI concludes the paper.

II. RELATED WORK

Smart contract vulnerability detection is becoming increasingly important due to the evolution of decentralized applications and the immutable nature of blockchain applications and transactions. Traditional detection methods have relied on analysis tools such as *Mythril*, *Slither*, and *SmartCheck*, which use rule-based pattern matching and symbolic execution [6]–[8]. These methods provide comprehensibility of results, however, the ability to detect complex vulnerabilities that are becoming more advanced is limited. It’s often seen that they generate false positives and struggle with deep semantic understanding of code structure and flaws.

Researchers have attempted to utilize various machine learning (ML) and deep learning (DL) techniques to address this. Vanilla Residual Neural Networks (RNNs), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and Graph Convolutional Networks (GCNs) have been applied to various smart contract datasets such as SCVDM-DLER and OSCV-MCEE [9], [10].

A key work in smart contract vulnerability detection is the *enhanced genetic algorithm (EGA) profiling model* that was developed to provide a transparent and feature-based approach to smart contract analysis in the BCCC-SCsVul-2024 data set from York University. [1]

- A large population size (10,000) for deep exploration of the solution space
- Semantic Similarity Distance (SSD) maintains diversity while also reducing the likelihood of implausible crossover combinations
- A penalty fitness function in order to improve the accuracy of results
- Biased initialization for simpler feature subsets and dynamic, adaptive mutation rates for maintaining extents of exploration and exploitation
- Elite retention to preserve high-quality candidates across generations.

The EGA Model [1] displays high explainability, as every vulnerability is mapped to different attributes from the source code. The model shows improvement from rule-based tools and traditional machine learning methods, but it relies heavily on handcrafted features, and evolutionary heuristics could hinder the capture of semantic and contextual information throughout the smart contracts in the dataset.

The difference in the approach presented in this paper is that SCRAN utilizes residual connections and multi-head attention mechanisms, which are further enhanced through ensemble learning and dynamic threshold tuning. It is technically less interpretable than the previous EGA used on the BCCC-SCsVul-2024 dataset. Still, the method achieves higher accuracy and robustness, which shows that there is a favorable trade-off between performance and explainability.

This paper is also compared to other works using similar methodologies on different smart contract vulnerability datasets.

III. DATASET AND PREPROCESSING

A. BCCC-SCsVul-2024 Dataset

The dataset contains 111,897 solidity-based smart contract samples, each represented by 240 static features extracted via control-flow analysis and opcode statistics. The dataset contains 11 different vulnerability types: ExternalBug, GasException, MishandledException, Timestamp, TransactionOrderDependence, UnusedReturn, WeakAccessMod, CallToUnknown, DenialOfService, IntegerUO, and Reentrancy, along with a non-vulnerability class (secure).

B. Data Cleaning and Splitting

Data is processed using pandas with error handling for malformed lines. All features are converted to numeric, and non-numeric entries are set to zero. The columns are standardized with z-score normalization:

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}, \quad (1)$$

μ_i and σ_i are the mean and standard deviation of feature i respectively, in the training batch of data.

The entire dataset undergoes a split into training (70%), validation (10%), and test (20%) subsets. Stratification isn’t used, so underrepresentation of classes is fought against. However, the test set does not undergo augmentation or threshold tuning for integrity and quality results.

C. SMOTE Oversampling

To address class imbalance, we apply SMOTE per class j when its positive count $n_j > 5$. Synthetic samples are generated using k -nearest neighbors, with sampling ratio $r_j = \min(0.3, 3n_j/N)$. Only synthetic samples are appended, each assigned a one-hot label for class j and zeros for others. This yields a more balanced training set while preserving multi-label structure.

Over each vulnerability class j , SMOTE is applied when its positive count $n_j > 5$ to reduce class imbalance. Synthetic samples are generated using k -nearest neighbors, with sampling ratio $r_j = \min(0.3, 3n_j/N)$. Only synthetic samples are appended, each assigned a one-hot label for class j and zeros for others. This helps to produce a balanced training set while preserving the structure of the dataset at the same time.

D. Probabilistic Data Augmentation

20,000 augmented samples were generated using these operations:

- Additive Gaussian noise: $\mathbf{x}_{aug} = \mathbf{x} + \mathcal{N}(0, 0.1^2 I)$.
- Random feature scaling: $\mathbf{x}_{aug} = \mathbf{x}_{aug} \odot s$, where $s \sim \mathcal{U}(0.9, 1.1)^{242}$.
- Feature dropout: apply dropout mask $d \sim \text{Bernoulli}(0.95)$, setting $x_{aug,i} = 0$ when $d_i = 0$.

When this is combined with original data, augmentation increases the robustness of the model with noisy/perturbed inputs.

E. Dynamic Class Weights

We compute per-class positive weights w_j for loss scaling:

$$w_j = \text{clip}\left(\sqrt{\frac{N - n_j + \epsilon}{n_j + \epsilon}} \times k_j, 1.0, 75.0\right), \quad (2)$$

and set $k_j = 1.5$ for classes with $\frac{N - n_j}{n_j} > 10$, and $k_j = 2.0$ when $n_j < 10$. Using such non-linear scaling penalizes the model's incorrect classification of less abundant classes.

IV. SCRAN ARCHITECTURE AND TRAINING

The **Smart Contract Residual Attention Network (SCRAN)** is our custom deep learning model that is designed to detect 11 distinct vulnerability classes in the BCCC-SCsVul-2024 smart contract dataset. The model utilizes residual fully connected networks, enhanced multi-head attention mechanisms, and class-specific attention heads in an ensemble learning framework. Three different versions of SCRAN are trained in the ensemble with varying hyperparameters to increase the robustness and efficiency of the model. Figure 1 provides an overview of the SCRAN architecture.

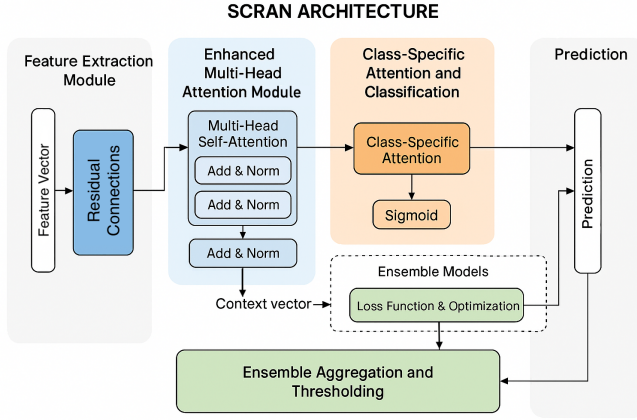


Fig. 1. SCRAN architecture.

A. Feature Extraction Module

The feature vectors or input to the model is a 240-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{240}$ from the static analysis of the dataset. The feature extractor consists of two residual fully connected layers, each followed by Layer Normalization, ReLU activation, and dropout for regularization.

$$\mathbf{h}^{(1)} = \text{LayerNorm}(W_1 \mathbf{x} + \mathbf{b}_1), \quad (3)$$

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{h}^{(1)}), \quad (4)$$

$$\mathbf{h}^{(1)} = \text{Dropout}(\mathbf{h}^{(1)}, p = 0.3), \quad (5)$$

$$\mathbf{h}^{(2)} = \text{LayerNorm}(W_2 \mathbf{h}^{(1)} + \mathbf{b}_2). \quad (6)$$

Residual connections are added in order to nurture gradient flow and improve the learning dynamics of the model. $\mathbf{x}$ is directly added if input and output dimensions match; otherwise, a learnable linear projection is used to align dimensions for the addition.

B. Enhanced Multi-Head Attention Module

This module augments the feature vectors with global contextual interactions. A positional encoding vector $\mathbf{p} \in \mathbb{R}^{1 \times d}$ is added to $\mathbf{h}^{(2)}$ as follows:

$$\tilde{\mathbf{h}} = \mathbf{h}^{(2)} + \mathbf{p}, \quad (7)$$

The vector $\tilde{\mathbf{h}}$ is projected into concatenated query, key, and value tensors via a multi-layer perceptron:

$$\mathbf{QKV} = W_{\text{qkv}} \tilde{\mathbf{h}}, \quad (8)$$

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \text{split}(\mathbf{QKV}). \quad (9)$$

The specific weights for attention are calculated using dot-product attention, which is scaled:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_h}}\right), \quad (10)$$

$$\mathbf{H} = \mathbf{AV}, \quad (11)$$

The outputs from above are passed through the residual normalization layers:

$$\mathbf{h}^{(3)} = \text{LayerNorm}(\mathbf{h}^{(2)} + \text{Dropout}(\mathbf{H})), \quad (12)$$

$$\mathbf{h}^{(4)} = \text{LayerNorm}(\mathbf{h}^{(3)} + \text{FFN}(\mathbf{h}^{(3)})), \quad (13)$$

Where $\text{FFN}(\cdot)$ feed-forward neural network with two layers and GELU activation functions.

C. Class-Specific Attention and Classification Module

For each vulnerability class $j \in \{1, \dots, 11\}$, a class-specific attention head computes a weight vector α_j over the feature dimensions:

$$\mathbf{a}_j = W_{j,2} \cdot \text{GELU}(W_{j,1} \mathbf{h}^{(4)} + \mathbf{b}_{j,1}) + \mathbf{b}_{j,2}, \quad (14)$$

$$\alpha_j = \text{softmax}(\mathbf{a}_j), \quad (15)$$

A context vector $\mathbf{c}_j$ is then computed by utilizing the attention weights as shown below:

$$\mathbf{c}_j = \alpha_j \odot \mathbf{h}^{(4)}, \quad (16)$$

Then, at last, a sigmoid classifier is used to label the output and identify the class j :

$$\hat{y}_j = \sigma(W_{j,c} \mathbf{c}_j + \mathbf{b}_{j,c}), \quad (17)$$

The complete output $\hat{\mathbf{y}} \in [0, 1]^{11}$ represents the probability of each vulnerability being present in the feature vector.

D. Loss Function and Optimization Strategy

SCRAN utilizes multi-label focal loss that is enhanced with class-specific weights and dynamic focusing parameters, as the dataset portrays severe class imbalance.

$$\mathcal{L}_m = - \sum_{j=1}^{11} w_j \cdot (1 - \hat{y}_j)^{\gamma_m} \cdot y_j \cdot \log(\hat{y}_j) \quad (18)$$

Each of the three ensemble models $m \in \{1, 2, 3\}$ uses a different focal loss focusing parameter $\gamma_m \in \{2.0, 2.5, 3.0\}$ for improved generalization and quality of results.

Optimization is performed using AdamW with weight decay 0.01 and learning rates $\{3 \times 10^{-5}, 3.6 \times 10^{-5}, 4.2 \times 10^{-5}\}$ for the different ensemble variants. For all the variants, training is performed over 150 epochs and a batch size of 32 with early stopping (patience of 15 epochs).

E. Ensemble Aggregation and Thresholding

Throughout training each ensemble variant, predictions from the three independently trained models are aggregated with class-wise weighted averaging of the logits. The binary predictions are finalized with the application of threshold tuning that is optimized with the precision-recall curve analysis. This ensures a tailored balance between precision and recall for each vulnerability's predictions.

F. Ensemble and Threshold Tuning

Validation set predictions from the three models are used to compute per-class ensemble weights:

The ensemble weights per class from the three independent models are calculated with the validation set predictions:

$$w_{j,m} = F1_{j,m}^2,$$

Ensemble logits are averaged using these weights. Decision thresholds τ_j are selected by scanning precision-recall curves to maximize F1 while maintaining recall ≥ 0.5 for classes with an imbalance ratio > 10 . The metrics are normalized across models.

V. EXPERIMENTAL RESULTS

A. Protocol and Results

On the test set (20% of the dataset), no SMOTE or augmentation is applied for fair results. The metrics produced are precision, recall, F1, and AUC. The confidence intervals for the macro metrics were calculated via t-distribution with 95% confidence.

Table I presents the detailed performance of the proposed model across twelve smart contract vulnerability classes, evaluated using five metrics: Accuracy, Precision, Recall, F1 Score, and AUC. The results demonstrate that the model achieves robust and consistent detection capabilities across most classes, with particularly strong performance in high-severity vulnerabilities.

The model performs exceptionally well on classes such as *CallToUnknown* (F1: 0.9684), *DenialOfService* (F1: 0.9737),

IntegerUO (F1: 0.9741), and *Reentrancy* (F1: 0.9681), all of which also exhibit high AUC scores exceeding 0.999. These results indicate the excellent discriminative ability and the extent of reliability that the model produces in detecting certain vulnerabilities. The classes also display high recall and precision, affirming the model's ability to identify and accurately predict the true vulnerabilities (true positives).

In contrast, the weakest performance of the model is observed in *Timestamp* (F1: 0.6752) and *TransactionOrderDependence* (F1: 0.7404), where lower precision suggests room for improvement, as there is a tendency towards false positives. This mishap could be attributed to the semantic ambiguity or the heavy class imbalance regarding these specific vulnerabilities. Patterns indicate areas where the model's attention mechanisms may benefit from refinement or rather targeted data augmentation to yield improvements.

The macro-average scores—Accuracy: 0.9873, Precision: 0.8069, Recall: 0.9572, F1 Score: 0.8703, and AUC: 0.9973—affirm the model's overall efficacy and reliability in spite of the impertinent class imbalance of the dataset. The recall and AUC being high display the model's ability of detecting most vulnerabilities without much omission, and the F1 score highlights a good balance between sensitivity and precision in classification.

To summarize the results, SCRAN shows promising classification metrics across all 11 different vulnerabilities with reliable generalization and high efficiency. An increase in precision on edge-case categories would increase robustness and improve detection further.

Figure 2 shows per-class precision, recall, and F1-scores.

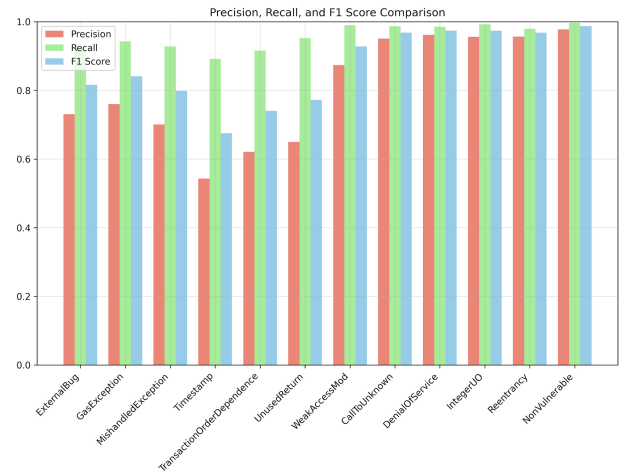


Fig. 2. Precision, Recall, and F1-score per vulnerability class

B. Comparison to Enhanced Genetic Algorithm

The Enhanced Genetic Algorithm (EGA) profiling model, proposed by HajiHosseinKhani *et al.* [1], uses evolutionary search to detect smart contract vulnerabilities in the BCCC-SCSVul-2024 dataset, focusing mainly on interpretability. The EGA uses semantic similarity checks, adaptive mutation, and

TABLE I
PER-VULNERABILITY PERFORMANCE OF PROPOSED MODEL

Vulnerability Type	Accuracy	Precision	Recall	F1 Score	AUC
ExternalBug	0.9866	0.7306	0.9254	0.8166	0.9967
GasException	0.9782	0.7598	0.9425	0.8414	0.9954
MishandledException	0.9786	0.7006	0.9277	0.7983	0.9940
Timestamp	0.9795	0.5434	0.8914	0.6752	0.9924
TransactionOrderDependence (TOD)	0.9791	0.6213	0.9161	0.7404	0.9942
UnusedReturn	0.9849	0.6497	0.9519	0.7723	0.9969
WeakAccessMod	0.9974	0.8736	0.9896	0.9280	0.9999
CallToUnknown	0.9934	0.9509	0.9865	0.9684	0.9995
DenialOfService	0.9942	0.9621	0.9857	0.9737	0.9995
IntegerUO	0.9920	0.9563	0.9926	0.9741	0.9996
Reentrancy	0.9898	0.9567	0.9799	0.9681	0.9994
NonVulnerable	0.9937	0.9775	0.9968	0.9871	0.9998
Macro Average	0.9873	0.8069	0.9572	0.8703	0.9973

elite retention for the evolution of feature subsets. This is very effective to find transparent patterns, however, the reliance on handcrafted features and limited representation depth restricts the ability of the model to generalize.

In comparison, SCRAN leverages deep residual connections, enhanced multi-head attention, and class-specific attention heads to automatically model latent patterns across all 11 vulnerability classes. It is further strengthened by SMOTE augmentation, class-weighted focal loss, and threshold tuning for robust multi-label detection.

As shown in Table II, SCRAN consistently outperforms the previously used EGA on the BCCC-SCSVul-2024 dataset:

- *External bug*: F1 improved from 0.61 (EGA) to 0.82 (SCRAN)
- *Re-entrancy*: F1 increased from 0.67 to 0.97
- *Integer overflow*: Accuracy improved from 0.83 to 0.99

SCRAN’s macro F1-score of 87.03% and macro accuracy of 98.73% surpasses that of the genetic algorithm’s 75% macro F1-score. SCRAN has an advantage in both precision and coverage, detecting rarer classes with more reliability.

C. Comparison on the Basis of Methodology

This sections compare SCRAN against four recent deep learning-based smart contract vulnerability approaches: EA-RGCN [11], MH-NEC [12], MEVD [13], and DeepFusion [14]. These models leverage semantic graphs, attention mechanisms, multi-branch encoders, or multimodal fusion to detect vulnerabilities in different smart contract datasets with similar methodologies to SCRAN.

Table III compares our model to EA-RGCN [11], which applies residual Graph Convolutional Networks (GCNs) with edge attention over semantic graphs. Chen et al. achieve 91.2% and 87.5% accuracy on *Reentrancy* and *Timestamp*, respectively, while SCRAN exceeds both with 98.9% and 97.9% accuracy, along with significantly higher F1 and AUC scores.

In Table IV, we compare with MH-NEC [12], which uses handcrafted feature graphs with node-edge attention fusion. The MH-NEC performs well on *Reentrancy* (F1 = 92.6%),

TABLE II
PERFORMANCE COMPARISON ON BCCC-SCSVul-2024 DATASET:
SCRAN VS. EGA PROFILING

Vulnerability Type	Method	Precision	F1-Score	Accuracy
External bug	SCRAN	0.73	0.82	0.99
	EGA	0.85	0.61	0.93
Gas exception	SCRAN	0.76	0.84	0.98
	EGA	0.82	0.64	0.88
Mishandled exception	SCRAN	0.70	0.79	0.98
	EGA	0.83	0.60	0.89
Timestamp	SCRAN	0.54	0.68	0.98
	EGA	0.84	0.58	0.94
Transaction order dependence	SCRAN	0.62	0.74	0.99
	EGA	0.88	0.61	0.93
Unused return	SCRAN	0.65	0.77	0.98
	EGA	0.87	0.56	0.93
Weak access modifier	SCRAN	0.87	0.93	0.99
	EGA	0.88	0.75	0.97
Call to unknown	SCRAN	0.95	0.97	0.99
	EGA	0.81	0.65	0.84
Denial of service	SCRAN	0.96	0.97	0.99
	EGA	0.79	0.62	0.83
Integer underflow/overflow	SCRAN	0.96	0.97	0.99
	EGA	0.83	0.75	0.83
Re-entrancy	SCRAN	0.96	0.97	0.99
	EGA	0.79	0.67	0.78
Secure (Non-vulnerable)	SCRAN	0.98	0.99	0.99
	EGA	0.94	0.96	0.99
Macro Average	SCRAN	0.81	0.87	0.98
	EGA	0.86	0.75	—

but poorly on *Access Control* (F1 = 61.3%). SCRAN improves both metrics to 96.8% and 92.8%, showing improvement in these vulnerability classes.

Table V displays SCRAN against MEVD [13], which combines a Transformer and Convolutional Neural Network (CNN) branches with a residual shrinkage network. MEVD achieves 91.0% F1 on *Timestamp* and 86.2% on *DoS*, while SCRAN produces 67.5% and 97.4% F1 respectively, with AUCs consistently above 0.99.

Finally, Table VI compares SCRAN with DeepFusion [14]. This approach fuses the predictions from source code, byte-code, and control flow modalities. Deg et. al have AUCs between 0.825 and 0.886, and SCRAN once again shows superiority across the different classes as AUCs are all 0.99

and above for the vulnerabilities on the table.

These comparisons showcase SCRAN’s superior ability to generalize and perform well against different smart contract vulnerability approaches as well as similar approaches used on different datasets despite only using static features. Ed models despite using only static features.

TABLE III
COMPARISON OF USER MODEL VS. CHEN ET AL. (2023)

Vulnerability	SCRAN Acc	Chen Acc
Reentrancy	0.9898	0.9120
Timestamp	0.9795	0.8750

TABLE IV
COMPARISON OF USER MODEL VS. HE ET AL. (2024)

Vulnerability	SCRAN F1	He F1
Reentrancy	0.9681	0.9255
Timestamp	0.6752	0.8463
WeakAccessMod	0.9280	0.6136

TABLE V
COMPARISON OF USER MODEL VS. GUO ET AL. (2024)

Vulnerability	SCRAN F1	Guo F1
Reentrancy	0.9681	0.8928
Timestamp	0.6752	0.9104
DenialOfService	0.9737	0.8621

TABLE VI
COMPARISON OF USER MODEL VS. DENG ET AL. (2023)

Vulnerability	SCRAN Acc	SCRAN AUC	Deng Acc	Deng AUC
Reentrancy	0.9898	0.9994	0.9090	0.8520
UnusedReturn	0.9849	0.9969	0.8950	0.8250
IntegerUO	0.9920	0.9996	0.9160	0.8340
TOD	0.9791	0.9942	0.9480	0.8860

D. Ablation Study

Evaluation of three ensemble SCRAN variants:

- Without focal loss: F1=0.8440, AUC=0.9951
- Single model (no ensemble): F1=0.8762, AUC=0.9964
- Without SMOTE+augmentation: recall on minority classes drops by 30%

These results confirm each component’s contribution.

VI. DISCUSSION

SCRAN is able to generalize across the different vulnerabilities due to the integration of data-level balancing while applying attention weights. Residual connections effectively stabilize the model’s architecture, and the class-specific attention focuses the model on the features that matter the most. Ensemble and threshold tuning help refine the training process and address the severe class imbalances.

The limitations of the proposed model include the utilization and reliance of static features. Using dynamic execution traces or graphical embeddings would capture semantics more effectively. Computational cost is not much and training the three models on a single GPU is about 12 hours elapsed.

VII. CONCLUSION

SCRAN presents a strong, efficient solution for multi-label smart contract vulnerability detection. Using the advanced preprocessing, comprehensive attention mechanisms, and ensemble learning, SCRAN has performed extremely well on the BCCC-SCsVul-2024 dataset.

SCRAN achieves a macro F1-score of 0.8703 and AUC of 0.9979, outperforming not only the Enhanced Genetic Algorithm (EGA) profiling model [1] but also four state-of-the-art deep learning baselines [11]–[14]. Across 11 vulnerability classes, SCRAN consistently yields higher accuracy, precision, and recall.

In the future, we hope to incorporate dynamic execution traces as well as graph-based semantic encoders for better interpretability and explainability. Understanding the dataset with better tools may have a lot to offer. Furthermore, it’s possible that unsupervised pre-training along with contrastive learning could reduce dependency on labeled data to enhance generalization for unseen vulnerabilities later on.

In conclusion, SCRAN demonstrates that attention-driven, ensemble-based architecture, when coupled with tailored data strategy, can significantly advance the precision and scalability of automated smart contract security analysis.

ACKNOWLEDGMENT

The author thanks Dr. Irfan Khan for insightful discussions and learning experiences throughout the MARE 491. The author also thanks graduate students Zahid Hussain and Shorya Agarwal for providing helpful assistance throughout the research process.

REFERENCES

- [1] S. HajiHosseiniKhan et al., “Unveiling Smart Contracts Vulnerabilities: Toward Profiling Smart Contracts Vulnerabilities using Enhanced Genetic Algorithm and Generating Benchmark Dataset,” *Blockchain: Research and Applications*, vol. 5, no. 2, p. 100253, 2024.
- [2] T.-Y. Lin et al., “Focal Loss for Dense Object Detection,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017.
- [3] H. Zhang et al., “ContractWard: Fine-Grained Vulnerability Detection for Ethereum Smart Contracts,” *IEEE Access*, vol. 10, pp. 75348–75358, 2022.
- [4] M. Chen et al., “Smart Contract Vulnerability Detection Using Attention Mechanisms,” in *Proc. Int. Conf. on Software Engineering (ICSE)*, 2022.
- [5] J. Li et al., “A Smart Contract Vulnerability Detection Method Based on Multimodal Feature Fusion and Deep Learning,” *Mathematics*, vol. 11, no. 23, 2023.
- [6] J. Krupp and C. Rossow, “teEther: Gnawing at Ethereum to Automatically Exploit Smart Contracts,” in *Proc. USENIX Security Symposium*, 2018.
- [7] T. Perez and A. Brent, “Slither: A Static Analysis Framework for Smart Contracts,” *Cryptic Labs*, 2019.
- [8] T. Luu et al., “Making Smart Contracts Smarter,” in *Proc. ACM Conf. on Computer and Communications Security (CCS)*, 2016.
- [9] R. A. Abdellatif et al., “SCVDM-DLER: A Dataset and Learning Framework for Smart Contract Vulnerability Detection,” in *Proc. IEEE Int. Conf. on Blockchain*, 2021.

- [10] D. Torres *et al.*, "OSCV-MCEE: A Multi-Context Evaluation Environment for Smart Contract Vulnerability Detection," in *Proc. IEEE Int. Conf. on Cyber Security and Protection of Digital Services*, 2022.
- [11] D. Chen, L. Feng, Y. Fan, S. Shang, and Z. Wei, "Smart Contract Vulnerability Detection Based on Semantic Graph and Residual Graph Convolutional Networks with Edge Attention," *Information Sciences*, vol. 646, pp. 119–134, 2023.
- [12] Z. He, Z. Zhao, K. Chen, and Y. Liu, "Smart Contract Vulnerability Detection Method Based on Feature Graph and Multiple Attention Mechanisms," *Computers, Materials & Continua*, vol. 78, no. 2, pp. 2163–2178, 2024.
- [13] J. Guo, L. Lu, and J. Li, "Smart Contract Vulnerability Detection Based on Multi-Scale Encoders," *Electronics*, vol. 13, no. 2, article 314, 2024.
- [14] W. Deng, H. Wei, T. Huang, C. Cao, Y. Peng, and X. Hu, "Smart Contract Vulnerability Detection Based on Deep Learning and Multimodal Decision Fusion," *Sensors*, vol. 23, no. 6, article 2924, 2023.